write `W`, or a sequence of atomic read-modify-write operations on `x` (such as `fetch_add()` or `compare_exchange_weak()`) by any thread, where the value read by the first thread in the sequence is the value written by `W` (see section 5.3.4).

Leave the "suitably tagged" part aside for now, because all operations on atomic types are suitably tagged by default. This essentially means what you might expect: if thread A stores a value and thread B reads that value, there's a synchronizes-with relationship between the store in thread A and the load in thread B, just as in listing 5.2.

As I'm sure you've guessed, the nuances are all in the "suitably tagged" part. The C++ memory model allows various ordering constraints to be applied to the operations on atomic types, and this is the tagging to which I refer. The various options for memory ordering and how they relate to the synchronizes-with relationship are covered in section 5.3.3. First, let's step back and look at the happens-before relationship.

5.3.2. The happens-before relationship

The *happens-before* relationship is the basic building block of operation ordering in a program; it specifies which operations see the effects of which other operations. For a single thread, it's largely straightforward: if one operation is sequenced before another, then it also happens-before it. This means that if one operation (A) occurs in a statement prior to another (B) in the source code, then A happens-before B. You saw that in listing 5.2: the write to `data` ❸ happens-before the write to `data_ready` ❹. If the operations occur in the same statement, in general there's no happens-before relationship between them, because they're unordered. This is just another way of saying that the ordering is unspecified. You know that the program in the following listing will output `"1, 2"` or `"2, 1"`, but it's unspecified which, because the order of the two calls to `get_num()` is unspecified.

Listing 5.3. Order of evaluation of arguments to a function call is unspecified

```
#include <iostream>

void foo(int a,int b)
{
    std::cout<<a<<","<<b<<std::endl;
}

int get_num()
{
    static int i=0;
    return ++i;
}

int main()
{
    foo(get_num(),get_num());
}
```

Calls to get_num() are unordered

There are circumstances where operations within a single statement are sequenced such as where the built-in comma operator is used or where the result of one expression is used as an argument to another expression. But in general, operations within a single statement are nonsequenced, and